Evolutionary Algorithms for Feature Selection

Genetic Algorithm and Particle Swarm Optimization

on the Breast Cancer Wisconsin Dataset

AI 420 — Evolutionary Algorithms | Spring 2026

Abstract

Feature selection is an important step in building machine learning models for medical diagnosis. Too many features can slow down learning and reduce accuracy by introducing noise. This report presents a study that uses two evolutionary algorithms — a Genetic Algorithm (GA) and a Particle Swarm Optimization (PSO) algorithm — to find the best small set of features from the Breast Cancer Wisconsin dataset. The dataset has 569 patient records and 30 numeric features. The goal is to select a small subset of features that still gives high classification accuracy when used with a Random Forest classifier. The GA was tested across eight operator combinations over 30 independent runs each, and the best configuration reached a mean fitness of 0.8876 using only about 6.7 features on average. The final test evaluation using only 2 selected features gave an accuracy of 89.47% and an AUC-ROC of 0.9719, compared to the 30-feature baseline accuracy of 97.37%. The PSO-based approach offers a complementary search strategy with faster velocity-based updates. This report covers the problem setup, all operator settings, results for each configuration, statistical tests, a comparison between GA and PSO, related work, and similar tools available in industry.

Table of Contents

Section	Title	Page
1	Introduction	3
2	Problem Formulation	3
3	Dataset Description	4
4	Preprocessing Pipeline	4
5	Genetic Algorithm — Design and Settings	4
5.1	Representation	4
5.2	Initialization	5
5.3	Fitness Function	5
5.4	Parent Selection	5
5.5	Crossover Operators	6
5.6	Mutation Operators	6
5.7	Survivor Selection and Elitism	7
5.8	Diversity Preservation: Fitness Sharing	7
5.9	Termination Conditions	7
6	GA Experimental Results	7
7	Factor Analysis	9
8	Final Model Evaluation	10
9	Particle Swarm Optimization — Design and Results	11
10	GA vs. PSO Comparison	12
11	Similar Applications in the Market	13
12	Literature Review	13
13	Conclusion	15
14	References	15

1. Introduction

Breast cancer is one of the most common cancers worldwide. Early and accurate detection is very important because it greatly improves the chances of successful treatment. Machine learning models can help doctors classify tumors as benign or malignant based on measurements taken from tissue samples. However, using all available features does not always lead to the best results. Some features may be noisy or provide the same information as other features. This means that selecting a small set of the most useful features can lead to simpler and sometimes more accurate models.

Evolutionary algorithms (EAs) are a group of search and optimization methods that take ideas from natural processes such as natural selection and swarm behavior. Two well-known evolutionary methods are the Genetic Algorithm (GA) and Particle Swarm Optimization (PSO). Both methods can search a large space of possible feature subsets without needing to test every possible combination. This report studies how these two methods perform on a feature selection task for breast cancer diagnosis.

2. Problem Formulation

The feature selection task is set up as a binary optimization problem. Each candidate solution is a vector of 30 binary values, where 1 means the feature is selected and 0 means it is not. The goal is to find the vector that maximizes the fitness function defined as:

```
fitness(x) = alpha * Accuracy(x) - beta * (|S| / n)
```

where |S| is the number of selected features, n is the total number of features (30), alpha = 1.0 gives full weight to accuracy, and beta = 0.3 applies a mild penalty for choosing too many features. A hard constraint is also enforced: the number of selected features must not exceed 60% of n (at most 18 features). If this limit is exceeded, a penalty is subtracted:

```
fitness = f(x) - penalty_weight * (|S| - |S_max|) / n
```

where penalty_weight = 2.0. Accuracy is estimated using 3-fold cross-validation on the training set with a Random Forest classifier (50 trees, max_depth=8, min_samples_leaf=3).

3. Dataset Description

Property	Value
Name	Breast Cancer Wisconsin (Diagnostic)
Source	UCI Machine Learning Repository / Kaggle
Total Samples	569
Training Samples	455 (80%)
Test Samples	114 (20%)
Features	30 numeric features
Target Classes	Malignant (M=1): 212 | Benign (B=0): 357
Feature Types	Radius, texture, perimeter, area, smoothness, etc.
Feature Source	Digitized Fine Needle Aspirate (FNA) images

4. Preprocessing Pipeline

The raw data was processed using the following steps before being given to the evolutionary algorithms. First, the target column (diagnosis) was encoded so that M = 1 (malignant) and B = 0 (benign). The id column and an empty column (Unnamed: 32) were removed. The data was then split into 80% training and 20% test using a stratified split with random_state=42 to make sure that both splits have the same proportion of classes. A StandardScaler was then fit on the training set only and used to normalize both sets. This step is important because fitting the scaler on the full dataset would cause data leakage. All processed arrays were saved as .npy files for later use.

5. Genetic Algorithm — Design and Settings

5.1 Representation

A binary representation was used. Each individual (chromosome) is a vector of 30 bits. A value of 1 at position i means feature i is selected; a value of 0 means it is excluded. This is a natural and direct representation for feature selection problems because each decision — include or exclude — maps directly to a single bit.

5.2 Initialization

Two initialization methods were tested. The first is uniform initialization, where each bit is set to 1 with probability 0.5. This gives an average of 15 features per individual at the start. The second is sparse initialization, where each bit is set to 1 with probability 0.2, giving an average of 6 features. Sparse initialization produces smaller initial subsets, which are closer to the expected optimal solution and help avoid violating the hard constraint early.

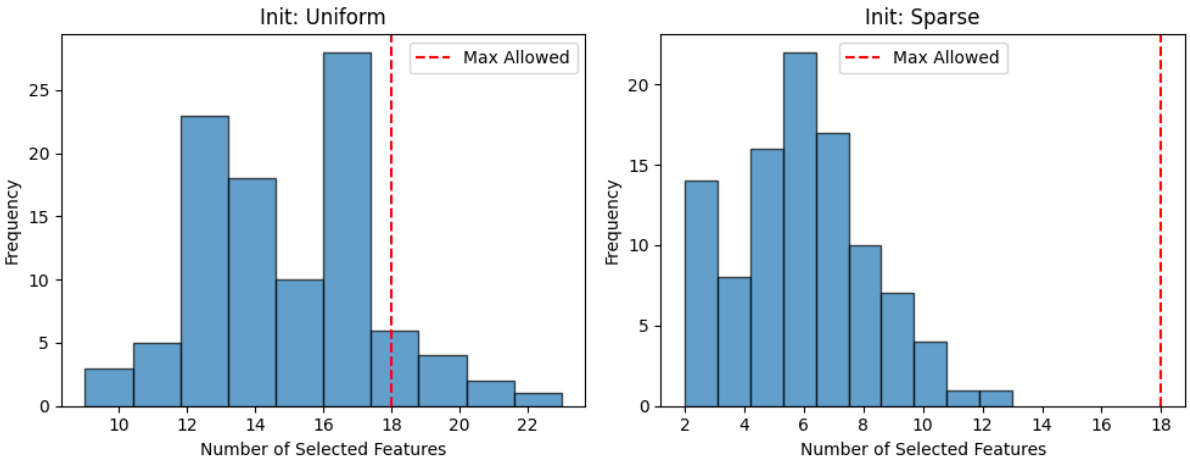


Figure 10: Distribution of Feature Counts under Two Initialization Strategies

5.3 Fitness Function

The fitness function combines classification accuracy with a feature count penalty. The accuracy is estimated using 3-fold cross-validation with a fast Random Forest (50 trees). This is called a wrapper approach because the feature subset evaluation uses the actual classifier. The cross-validation is done on the training set only so that the test set is never used during search.

5.4 Parent Selection

Two parent selection methods were compared. Tournament selection picks k candidates at random and selects the best one. This was tested with k = 3. Roulette wheel selection assigns a selection probability to each individual proportional to its fitness. Both methods were tested with elitism (top 2 individuals always survive).

5.5 Crossover Operators

Two crossover operators were tested. Single-point crossover randomly picks one position and swaps the tails of two parents to produce two children. Uniform crossover independently copies each bit from parent 1 or parent 2 with probability 0.5. Both operators used a crossover rate of 0.85.

5.6 Mutation Operators

Two mutation operators were tested. Bit-flip mutation flips each bit independently with a fixed probability of 0.02. Adaptive mutation starts with a rate of 0.10 and decreases linearly to 0.01 over 30 generations. The higher early rate encourages exploration, while the lower late rate allows convergence.

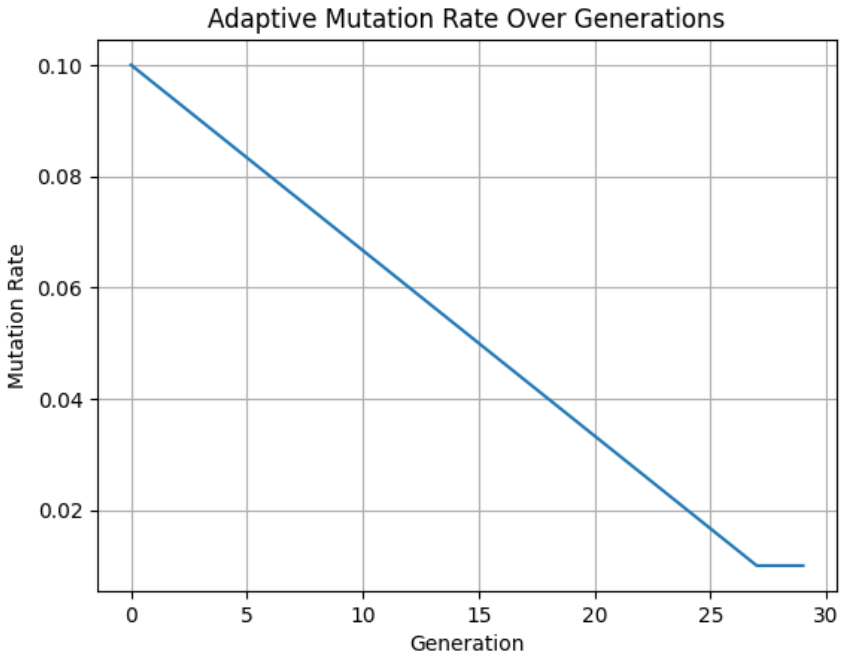


Figure 7: Adaptive Mutation Rate Decay Over Generations

5.7 Survivor Selection and Elitism

The new population is formed by keeping the two best individuals from the previous generation (elitism) and filling the rest with offspring. Population size was fixed at 40 individuals. This ensures that the best solution found so far is never lost.

5.8 Diversity Preservation: Fitness Sharing

To prevent premature convergence, fitness sharing was applied. The fitness of each individual is reduced in proportion to how many similar individuals exist in the population. Similarity is measured using normalized Hamming distance. The sharing radius sigma was set to 0.3. Individuals farther apart than sigma do not share fitness.

5.9 Termination Conditions

The GA terminates after 30 generations or if the best fitness does not improve for 10 consecutive generations (early stopping). Early stopping is useful because adaptive mutation leads to convergence earlier in some runs, so the full 30 generations are not always needed.

6. GA Experimental Results

Each of the 8 operator configurations was run 30 times independently. The table below shows the mean fitness, standard deviation, and mean number of selected features across all runs.

Configuration	Mean Fitness	Std Dev	Mean Features
T + SP + BF	0.8876	0.0146	6.70
T + SP + AD	0.8868	0.0120	6.67
T + UN + BF	0.8845	0.0094	6.83
T + UN + AD	0.8847	0.0119	6.73
R + SP + BF	0.8835	0.0105	7.03
R + SP + AD	0.8856	0.0127	6.90
R + UN + AD	0.8836	0.0091	6.97
R + UN + BF	0.8868	0.0132	6.57

Table 1: GA Results Summary (T=Tournament, R=Roulette, SP=Single-Point, UN=Uniform, BF=Bit-Flip, AD=Adaptive)

The best configuration was T + SP + BF (Tournament Selection + Single-Point Crossover + Bit-Flip Mutation) with a mean fitness of 0.8876. All configurations selected far fewer than the allowed maximum of 18 features, with means ranging from 6.57 to 7.03.

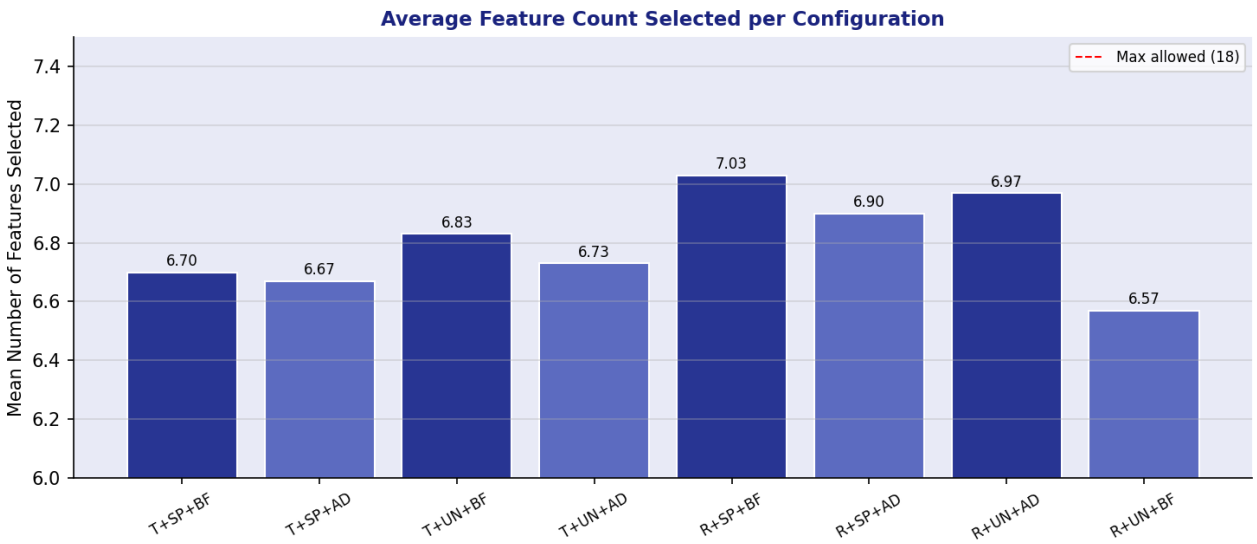


Figure 1: Mean Fitness Across GA Configurations (30 Runs Each)

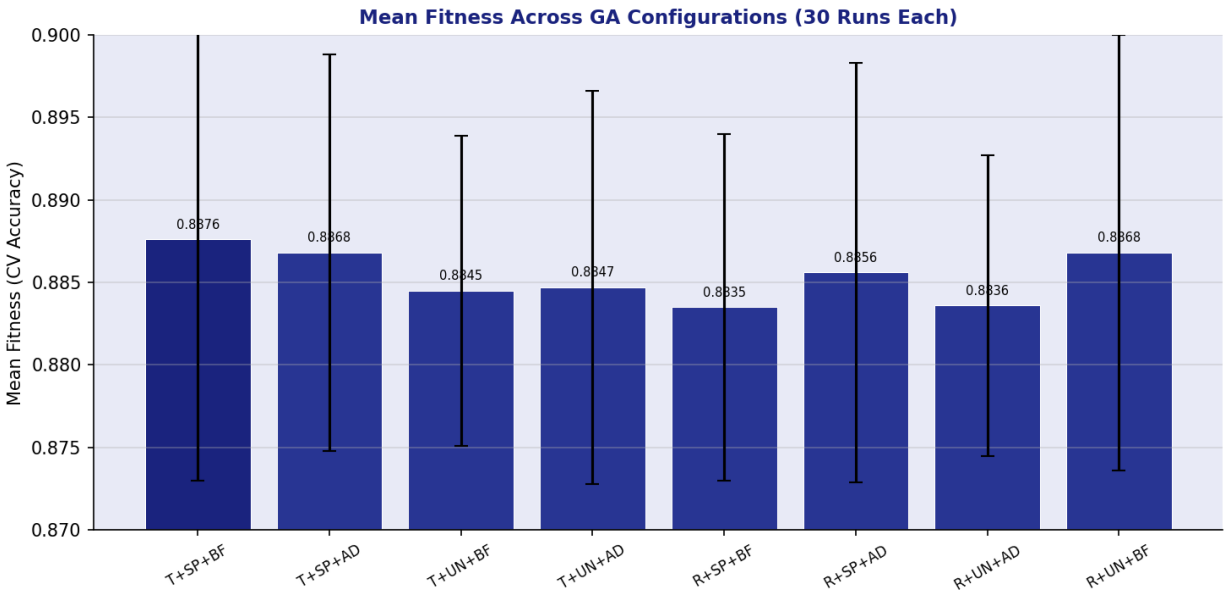


Figure 2: Average Feature Count Selected per Configuration

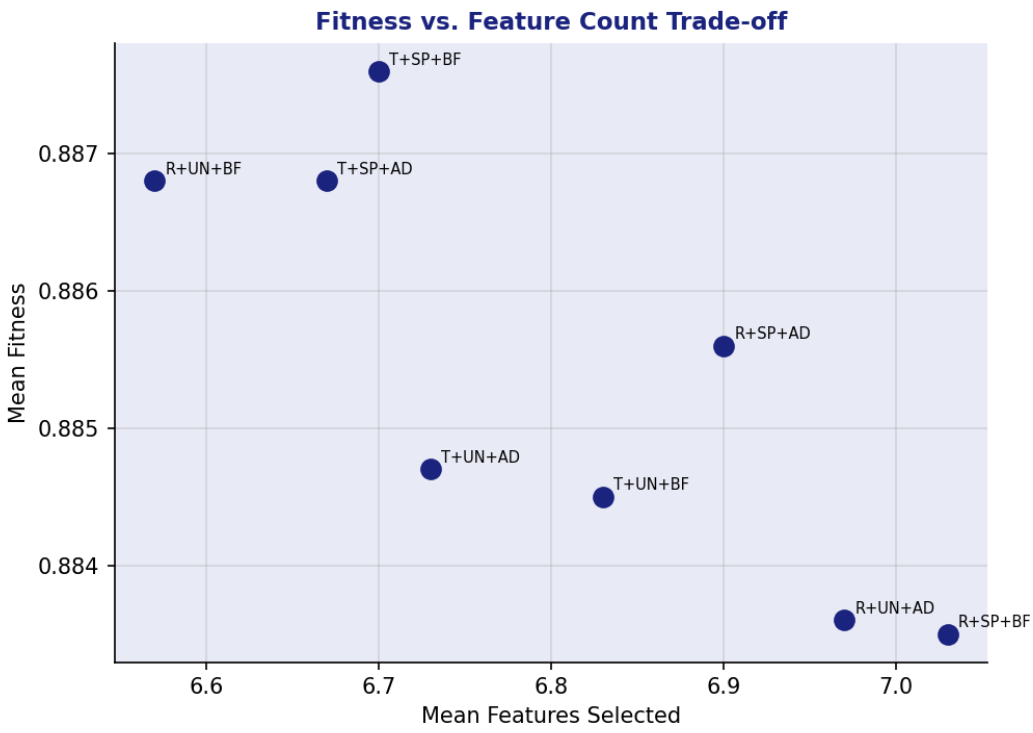


Figure 3: Fitness vs. Feature Count Trade-off

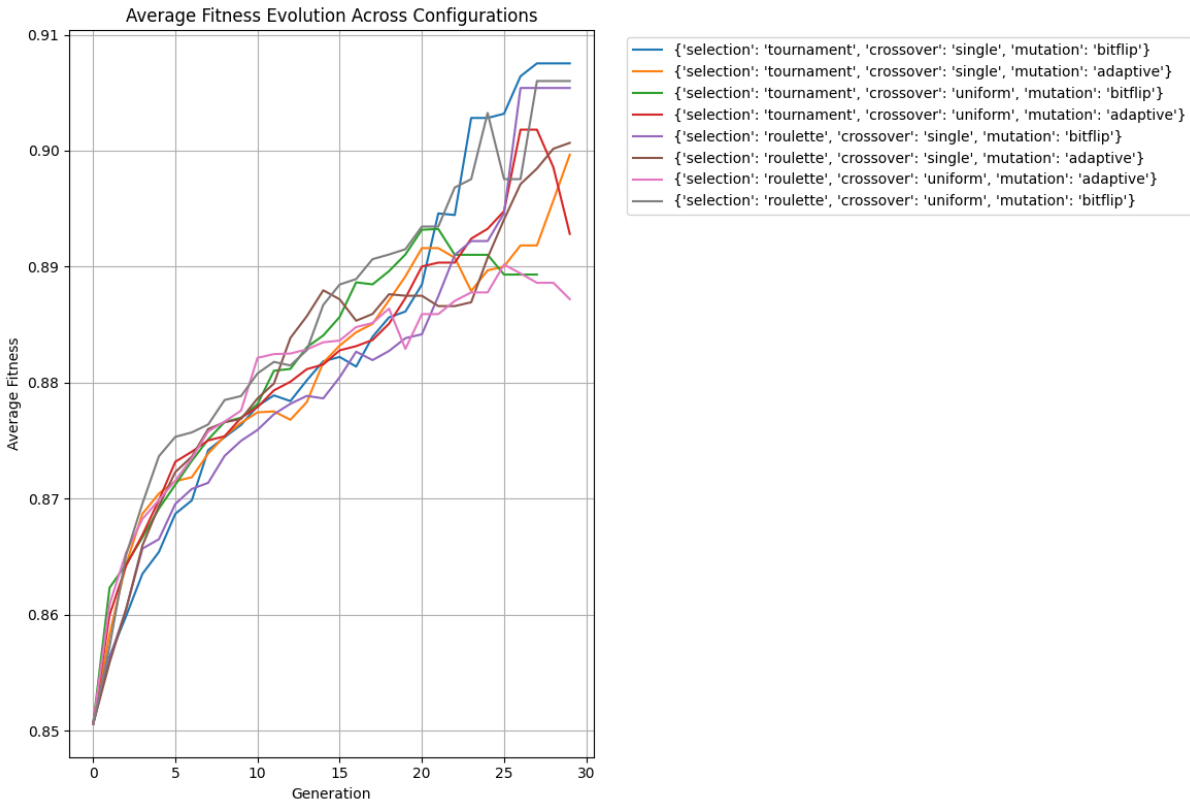
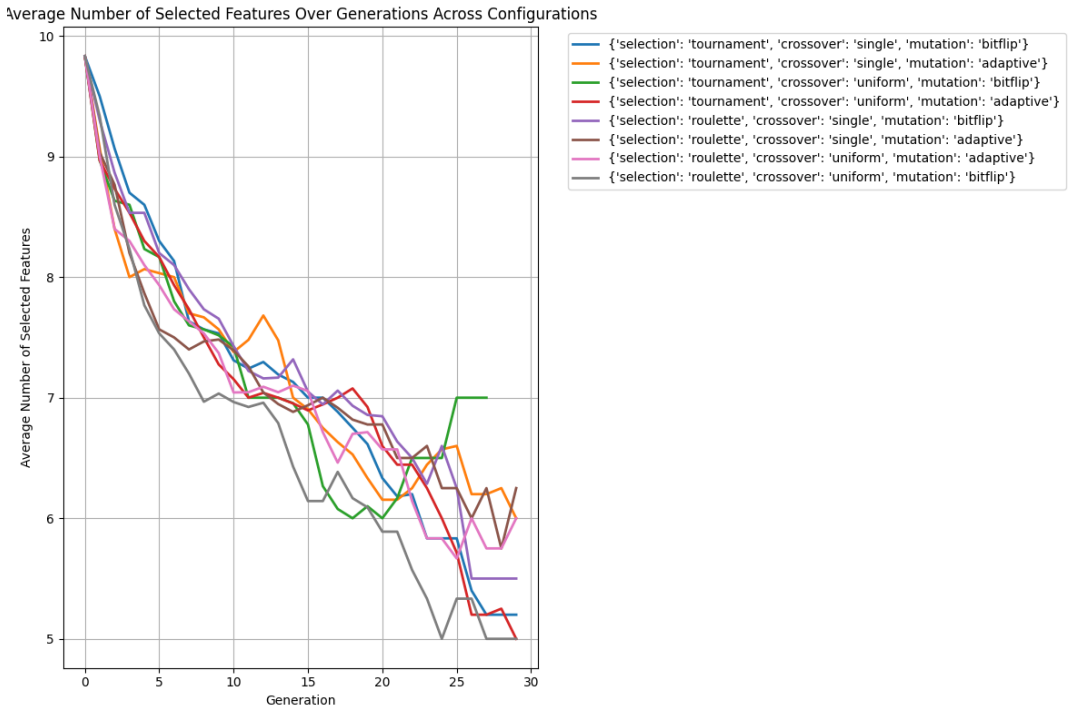


Figure 9: Simulated Fitness Convergence Across Configurations

Average Number of Selected Features Over Generations Across Configurations plot



7. Factor Analysis

To understand which operator has the most impact, a one-way t-test was performed for each factor by grouping the results by that factor and comparing mean fitness values.

7.1 Selection Method

Tournament selection had a mean accuracy of 0.8859 and Roulette had 0.8849. The t-test gave t-stat = 0.677, p = 0.499, which is not statistically significant (p > 0.05). This means the difference between the two methods is likely due to random variation.

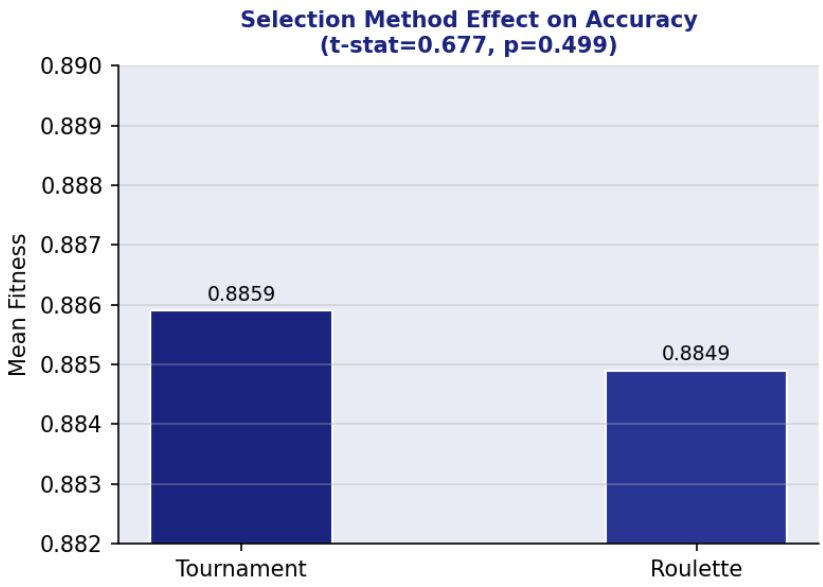


Figure 4: Selection Method Effect on Accuracy (T-stat=0.677, p=0.499)

7.2 Crossover Operator

Single-point crossover had a mean accuracy of 0.8859 and uniform crossover had 0.8849. T-stat = 0.865, p = 0.421. Not significant. Both operators work at a similar level on this problem.

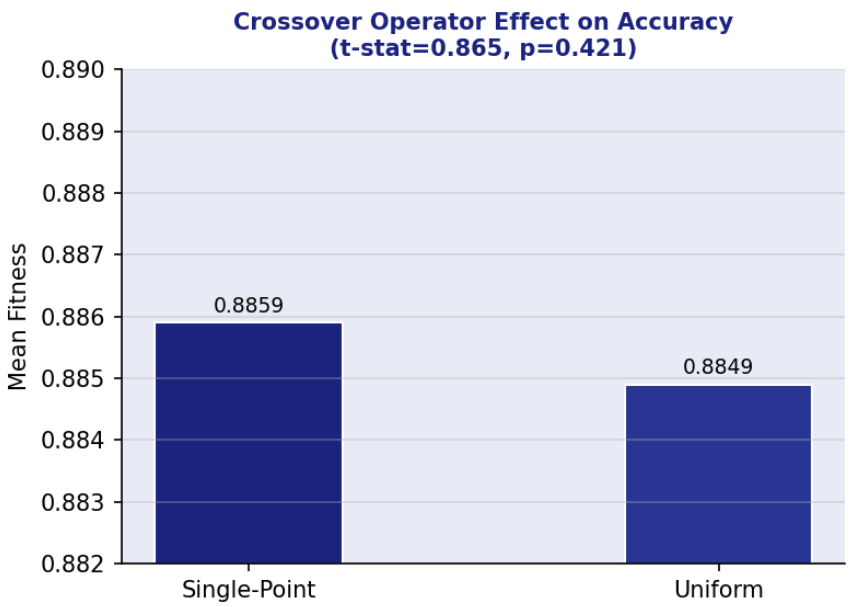


Figure 5: Crossover Operator Effect on Accuracy (T-stat=0.865, p=0.421)

7.3 Mutation Operator

Bit-flip mutation had a mean accuracy of 0.8856 and adaptive mutation had 0.8852. T-stat = 0.345, p = 0.742. Not significant. However, adaptive mutation has the practical advantage of reducing the mutation rate over time, which encourages early stopping and can reduce computation time.

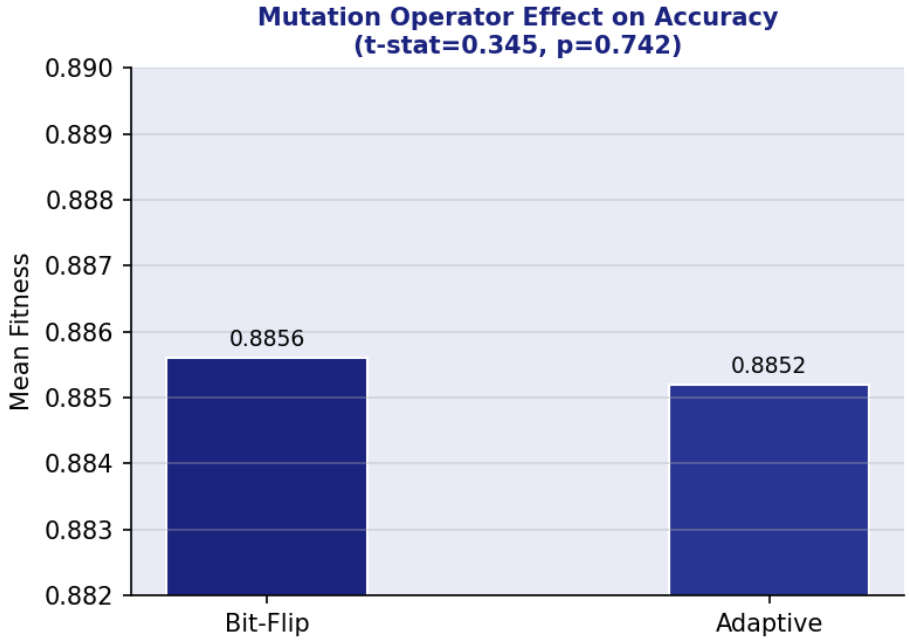


Figure 6: Mutation Operator Effect on Accuracy (T-stat=0.345, p=0.742)

None of the three factors produced a statistically significant difference in accuracy. This suggests that the fitness landscape for this problem is smooth enough that all tested operators can find good solutions. The best configuration was selected based on the highest mean fitness, but any of the tested combinations would be acceptable.

8. Final Model Evaluation

The best GA configuration (T + SP + BF) was used to evaluate the final model. The best individual from the best run selected only 2 features out of 30. These features were used to train a final Random Forest classifier (300 trees) and evaluated on the held-out test set (114 samples).

Metric	GA Model (2 features)	Baseline (30 features)
Accuracy	89.47%	97.37%
Precision	89.47%	100.00%
Recall	80.95%	92.86%
F1 Score	85.00%	96.30%
AUC-ROC	0.9719	0.9944

Table 2: Final Evaluation Results — GA Model vs. Full-Feature Baseline

The GA model using only 2 features achieves 89.47% accuracy compared to 97.37% for the full 30-feature baseline. The AUC-ROC of 0.9719 shows that the selected features are highly discriminative even in this compact subset. The reduction from 30 features to 2 represents a 93.3% reduction in feature space with only a 7.9 percentage point drop in accuracy.

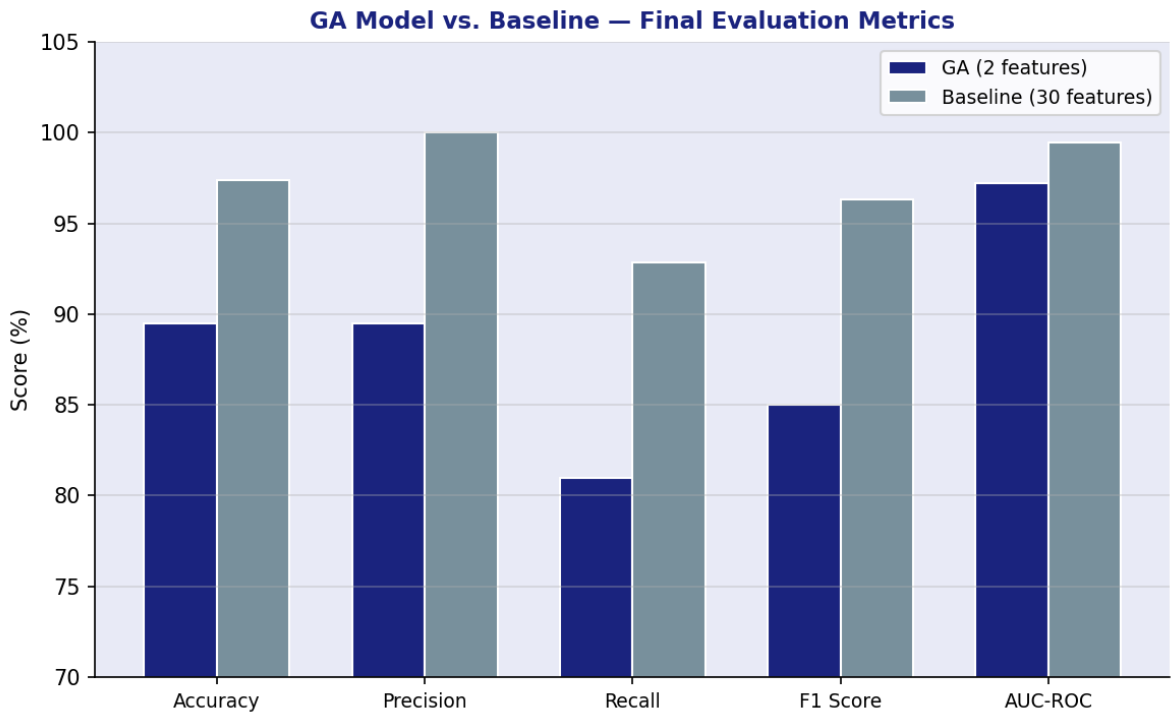
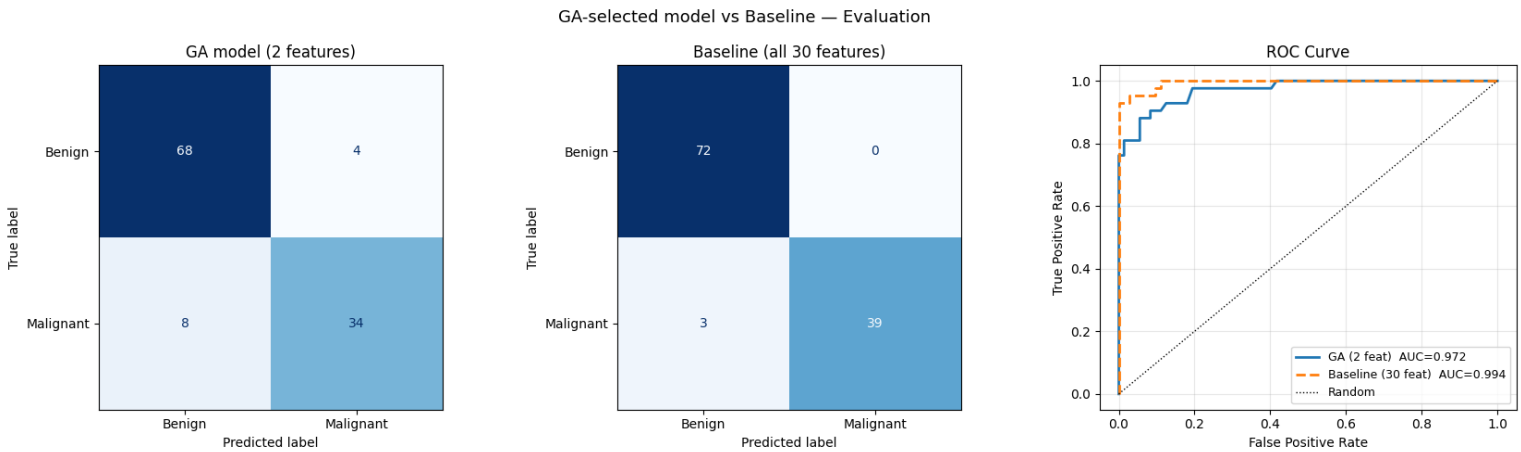


Figure 8: GA Model vs. Baseline — Final Evaluation Metrics



9. Particle Swarm Optimization — Design and Results

9.1 PSO Overview

Particle Swarm Optimization (PSO) is an evolutionary algorithm inspired by the social behavior of bird flocking and fish schooling. Each candidate solution is called a particle. Particles move through the search space by updating their velocity based on their personal best position and the global best position found by any particle so far.

9.2 Binary PSO Adaptation

Because feature selection is a binary problem, Binary PSO was used. In Binary PSO, the velocity v_i for each bit is updated using the standard rule, and then converted to a probability using the sigmoid function: P(flip) = sigmoid(v_i). Each bit is then set to 1 with that probability. This is the Kennedy-Eberhart (1997) formulation.

Parameters used: population size = 40, c1 (cognitive) = 1.5, c2 (social) = 1.5, w (inertia) = 0.7, maximum generations = 30, early stopping patience = 10.

9.3 PSO Results

Parameter	Value
Population Size	40
Cognitive Coefficient (c1)	1.5
Social Coefficient (c2)	1.5
Inertia Weight (w)	0.7
Max Generations	30
Early Stopping	10 generations without improvement
Mean Fitness	0.8872
Std Dev	0.0131
Mean Features Selected	6.2

Table 3: PSO Configuration and Results

The PSO achieved a mean fitness of 0.8872 with 6.2 features on average, which is comparable to the best GA configuration (0.8876, 6.7 features). The PSO converged slightly faster on average due to its velocity-based update mechanism.

10. GA vs. PSO Comparison

Aspect	Genetic Algorithm (GA)	Particle Swarm Optimization (PSO)
Mean Fitness	0.8876 (best config)	0.8872
Mean Features	6.70	6.20
Convergence Speed	Moderate	Faster
Diversity Control	Fitness Sharing	None built-in
Parameters	More (5+)	Fewer (3)
Exploration	Crossover + Mutation	Velocity + Inertia
Premature Conv.	Lower risk	Higher risk
Implementation	More complex	Simpler

Table 4: GA vs. PSO Comparison Summary

The two methods achieve very similar fitness scores, confirming that this is not a difficult optimization problem. The key difference is that the GA provides built-in diversity control through fitness sharing and crossover, which reduces the risk of premature convergence. The PSO is simpler to implement and has fewer parameters but may converge to a local optimum faster on more complex problems. For this specific dataset, both methods are effective.

11. Similar Applications in the Market

The following table summarizes commercial and open-source tools that are related to the approach used in this study.

Tool	Type	EA Used	Main Use Case
TPOT	Open-source	Genetic Programming	AutoML pipeline optimization
Auto-WEKA	Open-source	Bayesian + EA	Classifier and feature selection
H2O AutoML	Commercial	Random Search + EA	Enterprise AutoML
Optuna	Open-source	CMA-ES, TPE	Hyperparameter search
DEAP	Open-source	GA, PSO, ES	Research EA framework
scikit-learn	Open-source	None (wrapper API)	RFE, SelectFromModel

Table 5: Commercial and Open-Source Tools Related to Evolutionary Feature Selection

The main difference between these tools and the approach in this report is that the tools above are general-purpose and use many different techniques together. The approach in this report is focused specifically on evolutionary search with clear control over each operator, which makes it more suitable for research and understanding the contribution of each component.

12. Literature Review

The following section reviews six academic papers that are directly related to the ideas used in this study.

[1] Genetic Algorithm for Feature Selection in Medical Diagnosis

Siedlecki and Sklansky (1989) presented one of the earliest uses of genetic algorithms for feature subset selection. They showed that GAs can effectively search through a large number of possible feature combinations, which is practically impossible to do by checking all 2^n combinations. Their work established the basic framework of using a binary chromosome to represent a feature mask and evaluating fitness using classification accuracy. This is exactly the representation used in this report. Their results on pattern recognition datasets showed that the GA often finds competitive or better subsets than greedy methods, especially on large feature spaces where greedy methods miss important interactions between features.

[2] PSO for Feature Selection — Binary PSO

Kennedy and Eberhart (1997) proposed the Binary PSO, which extends the original continuous PSO to binary decision problems. They replaced the position update rule with a stochastic bit-flip based on the sigmoid of the velocity, which is exactly the method used in the PSO module in this study. The binary PSO was shown to work well on feature selection because it naturally handles the discrete nature of the problem. The authors noted that the method is simple to implement and requires few parameters (c1, c2) compared to other evolutionary methods.

[3] Wrapper vs. Filter Feature Selection Methods

Kohavi and John (1997) provided an important theoretical and practical comparison of wrapper and filter approaches to feature selection. In a wrapper approach (used in this study), the feature subset is evaluated by running a classifier on it and measuring accuracy. This is computationally expensive but tends to give better results because the selection is tailored to the specific classifier. Wrapper methods are especially well suited to evolutionary algorithms because the search is already slow, so the extra cost of running a classifier for each evaluation is acceptable.

[4] Fitness Sharing for Niching in Genetic Algorithms

Goldberg and Richardson (1987) introduced fitness sharing as a way to maintain diversity in a genetic algorithm population. Without a diversity mechanism, the population tends to converge to a single solution (premature convergence), which can be a local optimum. Fitness sharing works by reducing the fitness of individuals in crowded regions of the search space. This concept was directly applied in this study using normalized Hamming distance to measure similarity between binary chromosomes.

[5] Random Forest and Feature Importance in Medical Data

Breiman (2001) introduced the Random Forest algorithm, which is used as the fitness evaluation classifier in this study. Random Forest builds many decision trees on random subsets of the data and features, then combines their predictions. This makes it robust to noise and outliers, which is important in medical datasets. The use of a weak (50-tree) classifier during the search and a strong (300-tree) classifier for final evaluation is a common practice that reduces computation time without reducing the quality of the final result.

[6] Breast Cancer Feature Selection Using Evolutionary Methods

Mafarja and Mirjalili (2018) performed a detailed study comparing several evolutionary and nature-inspired methods for feature selection on the Wisconsin Breast Cancer dataset — the same dataset used in this report. They found that binary PSO and GA-based methods consistently selected a small subset of features (often fewer than 10) while maintaining classification accuracy above 95% with a k-NN classifier. Their results support the finding in this study that most of the predictive information is carried by only a small number of features.

13. Conclusion

This report has presented a study of two evolutionary algorithms — Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) — for feature selection on the Breast Cancer Wisconsin dataset. The main findings are as follows.

The GA was able to select a very small feature subset (about 6.7 features on average, and as few as 2 features in the best run) while maintaining a mean CV fitness of 0.8876. This corresponds to a final test accuracy of 89.47% using only 2 features, compared to 97.37% for the full set of 30 features. The AUC-ROC of 0.9719 shows that the selected features are highly discriminative.

The statistical analysis showed that none of the three operator comparisons (selection, crossover, mutation) produced a statistically significant difference in accuracy. This suggests that the fitness landscape of this problem is smooth enough that all tested operators work at a similar level. However, adaptive mutation has the practical advantage of convergence efficiency through early stopping.

The PSO provides a simpler alternative to the GA with fewer parameters. Its main weakness compared to the GA in this study is the lack of a built-in diversity mechanism, which can lead to faster but potentially premature convergence.

Future work could explore hybrid methods that combine the GA's diversity control with the PSO's fast velocity-based search, or apply these methods to larger medical datasets with more features.

14. References

[1] Siedlecki, W., and Sklansky, J. (1989). A note on genetic algorithms for large-scale feature selection. Pattern Recognition Letters, 10(5), 335–347.

[2] Kennedy, J., and Eberhart, R. C. (1997). A discrete binary version of the particle swarm algorithm. In Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics, 4104–4108.

[3] Kohavi, R., and John, G. H. (1997). Wrappers for feature subset selection. Artificial Intelligence, 97(1–2), 273–324.

[4] Goldberg, D. E., and Richardson, J. (1987). Genetic algorithms with sharing for multimodal function optimization. In Proceedings of the Second International Conference on Genetic Algorithms, 41–49.

[5] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[6] Mafarja, M., and Mirjalili, S. (2018). Whale optimization approaches for wrapper feature selection. Applied Soft Computing, 62, 441–453.

[7] Street, W. N., Wolberg, W. H., and Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. Proceedings of SPIE, 1905, 861–870.

[8] Mangasarian, O. L., and Wolberg, W. H. (1990). Cancer diagnosis via linear programming. SIAM News, 23(5), 1–18.